

FINLEDGER · ACCOUNTS PAYABLE AUTOMATION

Step-by-step plan

Two tracks run together: the **product build** (four people) and the **Antigravity loop** (Opus plans, Gemini Pro builds).

A. How the four humans work

Day 0 — freeze

- Put this folder in the repo as docs/ + context/ + .agents/.
- All four read docs/PRODUCT.md, docs/HOW_IT_WORKS.md, shared/00_SHARED_CONTRACTS.md.
- Lock JSON schemas for CanonicalInvoice, RiskScore, VoucherDraft, CorrectionEvent, MapTrace in packages/contracts/.
- Agree queue names: ingest | extract | score | map | post.
- Paste architecture refusals into README.md.

Weeks 0–4 — Tally loop only

Week	Person 1	Person 2	Person 3	Person 4
0	Schemas, router, token log, 20 bills	Tenancy, store, link, email	State enum, PDF queue shell	Tally hello + company read
1	Digital extract → CanonicalInvoice	Markdown + page images	Deterministic risk	fetch_masters
2	RAG + mapper → VoucherDraft	Masters write path	3-col UI + CorrectionEvent	XML preview, no post
3	Vision path, AI paragraph, eval	Queue retries	Approve draft + SoD	Post on approve
4	Correction memory live	Ageing storage	MSME / 180-day clocks	Idempotent post, DN/CN

Stop after week 4 and run a real CA client through Tally. Do not start Zoho, SAP, WhatsApp, or payments until that loop is boring.

Daily

- 15 min standup: blocked handoff only.
- Same 10 fixture bills for everyone.
- Friday: \$ / bill, % needing review, % first-try posts.

B. How Antigravity is used (cost control)

Opus 4.6 = planner only. Writes plans and reviews diffs. Does not implement 40 files.

Gemini Pro = workers. Implement one milestone each, isolated context.

Shared markdown = the only memory between humans and agents. If it is not in docs/ or context/, it does not exist for the next session.

Every Person 1 session

- Open Antigravity on the repo.
- Select Opus 4.6 as the **main agent**.
- Paste prompts/ANTIGRAVITY_PERSON1_RAG.md (the prompt in this pack).
- Opus must first write/update: - docs/plans/person1_implementation_plan.md - docs/plans/person1_task_list.md
- You comment on the plan artifact. Approve only then.
- Tell Opus: spawn Gemini Pro subagents from .agents/agents/ for each task. One task per subagent. model: pro.
- Each subagent reads context/PERSON_1.md + docs/HOW_IT_WORKS.md + the contracts, then writes code + updates context/PERSON_1_LOG.md.
- Opus reviews the diff against refusals. If a worker emitted Tally XML or a global RAG, reject.

Why this is cheaper

- Opus sees plans, contracts, and diffs — not every extract fixture.
- Pro sees one worker charter (schemas, or retriever, or eval) and a short context pack.
- Humans leave breadcrumbs in context/*.md so the next session does not re-read the whole architecture.

C. Context files the four people keep live

File	Who updates	After every work block
context/PERSON_1.md	Person 1	What shipped, current models, open questions for P2/P3
context/PERSON_2.md	Person 2	Table names, queue URLs, object path examples
context/PERSON_3.md	Person 3	State enum, policy flags, UI routes
context/PERSON_4.md	Person 4	Agent pairing, mapping table columns, sample XML
context/DECISIONS.md	whoever decided	Date + decision + why
context/HANDOFFS.md	producer	Payload that just changed

Rule: append, do not rewrite history. New session starts by reading PRODUCT + HOW_IT_WORKS + your PERSON file + DECISIONS + HANDOFFS.